
Recuperação de Informação

Aula 01 — Do problema da busca ao nosso motor

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Agenda de hoje

- 1 O que é Recuperação de Informação (RI)
- 2 O problema da busca em bases on-line
- 3 O projeto do semestre: um motor de busca
- 4 A arquitetura moderna (retrieve & rerank)
- 5 Por que usar **R**
- 6 Mãos à obra: primeiro *corpus* em R
- 7 Tokenização e frequência de termos
- 8 Matriz termo-documento e uma busca simples

Meta da aula

Entender o *que* vamos construir e escrever nossas *primeiras linhas de R* para indexar textos.

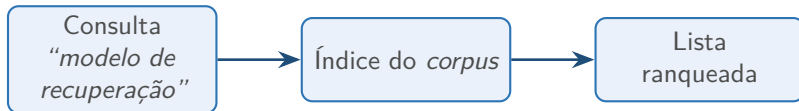
O que é Recuperação de Informação?

Definição

Recuperação de Informação — RI, em inglês *Information Retrieval* (IR) — é a área que trata de **encontrar documentos relevantes** dentro de grandes coleções, a partir de uma **necessidade de informação** expressa por uma *consulta*.

- Não é busca exata em banco de dados: lidamos com **relevância**, não com igualdade.
- O resultado é uma **lista ranqueada** — os melhores documentos primeiro.
- É o coração de mecanismos de busca, e-commerce, **RAG** (*Retrieval-Augmented Generation*) e assistentes de inteligência artificial.

O problema: consulta → documentos relevantes



- Precisamos **representar** texto de forma que o computador possa comparar.
- Precisamos de um **modelo de ranqueamento** que ordene por relevância.
- Precisamos **avaliar** se o ranking é bom (próximas aulas).

Objetivo

Construir, do zero, um **motor de busca** sobre um *corpus* (a coleção de documentos) on-line — e evoluir dos modelos clássicos até técnicas neurais e busca por **fórmulas matemáticas** (**MIR**, *Mathematical Information Retrieval*).

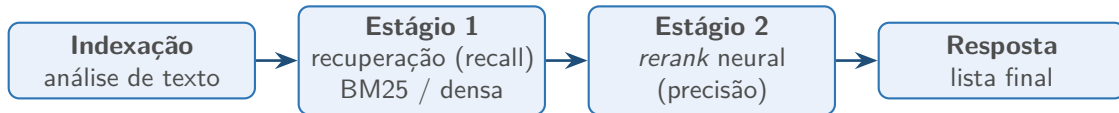
Vamos passar por:

- Análise de texto e índice invertido
- Booleano, Espaço Vetorial, **BM25** (*Best Matching 25*)
- Recuperação densa e *rerank* neural
- Busca por fórmulas (MIR)
- Aplicação web + avaliação

Integra com:

- Inteligência Computacional
- Linguagem e seus Códigos II
- Teoria do Aprendizado Estatístico

Arquitetura moderna: *retrieve & rerank*



- *Retrieve & rerank* (“recuperar e reordenar”): estágios **baratos** filtram muitos documentos; estágios **caros e precisos** refinam poucos.
- Começamos hoje pela **base**: transformar texto em algo indexável.

Por que R nesta disciplina?

- Linguagem **estatística** por natureza — conversa direto com Teoria do Aprendizado Estatístico.
- Ótimo para **prototipar** e inspecionar dados interativamente.
- Ecossistema de texto maduro: `tm`, `quanteda`, `tidytext`, `text2vec`.
- Vetores e matrizes são **cidadãos de primeira classe** — ideal para modelos de RI.

Nesta aula

Usaremos apenas **R base** para desmistificar cada passo. Depois adotamos pacotes especializados.

Vamos criar uma coleção de 8 documentos curtos para experimentar.

```
docs <- c(  
  d1 = "recuperacao de informacao ordena documentos por relevancia",  
  d2 = "o modelo de espaco vetorial representa documentos como vetores",  
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",  
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",  
  d5 = "o indice invertido acelera a busca em muitos documentos",  
  d6 = "embeddings capturam a semantica de palavras e documentos",  
  d7 = "a avaliacao mede a relevancia dos resultados da busca",  
  d8 = "ciencia de dados combina estatistica e programacao"  
)  
length(docs)  
docs["d5"]
```

```
[1] 8
```

```
d5
```

```
"o indice invertido acelera a busca em muitos documentos"
```

Tokenização: do texto aos termos

Tokenizar é quebrar o texto em unidades (termos, ou *tokens*). Aqui, por espaços.

```
tokenizar <- function(texto) {  
  texto <- tolower(texto)  
  unlist(strsplit(texto, "\\s+"))  
}  
tokens <- lapply(docs, tokenizar)  
tokens[["d1"]]  
  
[1] "recuperacao" "de"          "informacao"  "ordena"  
[5] "documentos"  "por"          "relevancia"
```

O **vocabulário** é o conjunto de termos distintos do *corpus*.

```
vocab <- sort(unique(unlist(tokens)))  
length(vocab)  
  
# frequencia total de cada termo no corpus  
freq <- table(unlist(tokens))  
sort(freq, decreasing = TRUE)[1:6]  
[1] 45
```

```
de a documentos e busca modelo  
6 5          4 3      2      2
```

A **matriz termo-documento** (**TDM**, *Term-Document Matrix*) conta quantas vezes cada termo aparece em cada documento.

```
tdm <- sapply(tokens, function(tk) {  
  as.integer(table(factor(tk, levels = vocab)))  
})  
rownames(tdm) <- vocab
```

```
tdm[1:6, ]      # um recorte: 6 primeiros termos x 8 documentos
```

	d1	d2	d3	d4	d5	d6	d7	d8
a	0	0	0	1	1	1	2	0
acelera	0	0	0	0	1	0	0	0
aprendizado	0	0	0	1	0	0	0	0
avaliacao	0	0	0	0	0	0	1	0
bm25	0	0	1	0	0	0	0	0
busca	0	0	0	0	1	0	1	0

Uma busca booleana simples

Recuperar todos os documentos que contêm o termo da consulta.

```
busca_booleana <- function(termo, tdm) {  
  termo <- tolower(termo)  
  if (!termo %in% rownames(tdm)) return(character(0))  
  colnames(tdm)[tdm[termo, ] > 0]  
}  
  
busca_booleana("documentos", tdm)  
busca_booleana("busca", tdm)  
[1] "d1" "d2" "d5" "d6"  
[1] "d5" "d7"
```

Simples, mas sem **ordenação**: todos “valem o mesmo”. Daí precisarmos de ranqueamento.

Um passo rumo ao ranqueamento: TF-IDF

TF-IDF — *Term Frequency* (frequência no documento) $\times$ *Inverse Document Frequency* (raridade no *corpus*) — pondera termos: frequentes *no documento*, porém raros *no corpus*, pesam mais.

```
N <- ncol(tdm)
df <- rowSums(tdm > 0)           # em quantos docs cada termo aparece
idf <- log(N / df)              # inverse document frequency
tfidf <- tdm * idf              # recicla idf por linha

round(tfidf[c("documentos", "recuperacao", "busca", "de"), ], 2)
```

	d1	d2	d3	d4	d5	d6	d7	d8
documentos	0.69	0.69	0.00	0.00	0.69	0.69	0.00	0.00
recuperacao	1.39	0.00	0.00	1.39	0.00	0.00	0.00	0.00
busca	0.00	0.00	0.00	0.00	1.39	0.00	1.39	0.00
de	0.47	0.47	0.94	0.00	0.00	0.47	0.00	0.47

Do brinquedo ao real: um *corpus* da Wikipédia

Nossos 8 documentos servem para **enxergar as contas**. Mas o motor de verdade precisa de texto real — e a Wikipédia é uma fonte aberta e fácil de obter.

```
library(httr2)

baixar_wiki <- function(titulo) {
  request("https://pt.wikipedia.org/w/api.php") |>
    req_url_query(action = "query", prop = "extracts", explaintext = 1,
                  format = "json", redirects = 1, titles = titulo) |>
    req_perform() |> resp_body_json() |>
    (\\(r) r$query$pages[[1]]$extract)()
}

texto <- baixar_wiki("Santos (São Paulo)")
substr(texto, 1, 60)
```

A licença é **CC BY-SA**: pode usar, desde que **cite a fonte**.

Por que a Wikipédia é um bom *corpus*

Vantagens

- Licença **aberta** (CC BY-SA)
- Texto **limpo** e estruturado
- Em português, sobre a **nossa região**
- Tem **links** entre artigos (útil adiante)

Sugestão de corpus

- Os **9 municípios** da Baixada Santista
- Mais o artigo do **Porto de Santos**
- $\approx$ 10 documentos longos para indexar

Atenção

Textos reais são **longos e sujos**: vão exigir a limpeza que veremos na Aula 03.

Sigla	Significado
RI / IR	Recuperação de Informação (<i>Information Retrieval</i>)
TDM	<i>Term-Document Matrix</i> — matriz termo-documento
TF	<i>Term Frequency</i> — frequência do termo no documento
IDF	<i>Inverse Document Frequency</i> — raridade no <i>corpus</i>
BM25	<i>Best Matching 25</i> — modelo probabilístico de ranqueamento
MIR	<i>Mathematical Information Retrieval</i> — busca por fórmulas
RAG	<i>Retrieval-Augmented Generation</i> — LLM fundamentado em busca
LLM	<i>Large Language Model</i> — modelo de linguagem de grande porte

Lembrando a Aula 00: regex casa *pedaço*

Na Aula 00 vimos grep e as *regex*. Vamos compará-las com o nosso índice.

```
grep("recupera", docs)          # acha "recuperacao": casa SUBSTRING
busca_booleana("recupera", tdm)  # nao acha: o TERMO nao existe

[1] 1 4
character(0)
```

A lição

A regex enxerga o texto como uma **sequência de caracteres**; nosso motor enxerga como um **conjunto de termos**. Por isso tokenizamos — e por isso um motor de busca é mais que uma busca por padrão.

Tarefa principal

Para **cada** bloco de código desta aula:

- ① **Explicar**: escreva, com suas palavras, o que cada linha faz e por que ela existe.
- ② **Explorar**: rode o código, **altere** algo (um documento, um termo, um argumento) e registre *o que mudou no resultado*.
- ③ **Prever**: antes de rodar a versão alterada, escreva o que você *espera* que aconteça — e compare.

Entregue um documento curto com o código, sua explicação e as saídas observadas.

Monte o *corpus* da Baixada Santista

- 1 Baixe da Wikipédia os artigos de **3 municípios** da região (comece pequeno).
- 2 Guarde-os num vetor nomeado docs, como fizemos em aula.
- 3 Tokenize, monte o **vocabulário** e responda: quantos termos distintos surgiram? Compare com os **45** do nosso *corpus* de brinquedo.
- 4 Liste os **10 termos mais frequentes**. Eles são *informativos*? O que isso sugere sobre a próxima aula?

Cite a fonte (CC BY-SA). Esse *corpus* será a base do seu motor no restante do semestre.

Próxima aula

- Modelo do Espaço Vetorial
- Similaridade do cosseno e ranking

Leitura

- Baeza-Yates & Ribeiro-Neto — cap. 1–3
- Manning et al., *IIR* — cap. 1

Guarde esta ideia

Hoje o motor só diz se um termo aparece. Na próxima aula ele passa a dizer *o quanto* o documento combina com a consulta.

Obrigado!

Dúvidas? Vamos ao R.
